

Informe Técnico: Taller de RxJS

(Semana 9)

Programación Reactiva y Patrón Publish/Subscribe • Desarrollo Web • Pontificia
Universidad Javeriana

DOCUMENTO DE
EQUIPO

1. ¿Qué se debía hacer en la Semana 9?

El objetivo de la Semana 9 es preparar el terreno para la conexión real entre el Frontend (Angular) y el Backend (Spring Boot), la cual se formalizará en la Semana 10 mediante servicios REST.

Como los navegadores y las llamadas de red son **asíncronas**, no es posible programar como si los datos estuvieran disponibles de inmediato en memoria. En el video de la clase (https://youtu.be/i00m3f_mDao) y en la presentación oficial ([SEMANA 9.RXJS-1.pdf](#)), el profesor Juan Sebastián Angarita solicitó comprender y desarrollar:

1. **Repaso de funciones en JavaScript:** Arrow functions, ventajas de declarar con `const` frente a `function`, y la solución del contexto léxico de `this`.
2. **Programación Reactiva pura con Node.js:** Fundamentos de RxJS, el rol de los Observables y Observers, operadores de creación, filtrado y transformación con tuberías (`pipe`), y operadores de combinación (`merge` vs `concat`).
3. **Consumo de APIs en Angular con RxJS:** Cómo inyectar `HttpClient`, consumir la API pública `JSONPlaceholder`, y resolver dependencias de datos asíncronas con `mergeMap` evitando condiciones de carrera.

¿Por qué no había repositorio oficial? El profesor construyó todo el código en vivo durante la sesión grabada de 95 minutos. El taller consiste en reproducir, probar y documentar esa misma arquitectura paso a paso.

2. Estructura del Taller Implementado (TallerRXJS)

El taller se organizó de forma modular en tres carpetas claramente separadas e independientes:

Directorio	Archivos Clave	Temas que Cubre
1.Funciones/	<code>funciones.js</code> <code>arrow_vs_normal.js</code>	Hoisting, funciones tradicionales vs flecha, retorno implícito, callbacks y <code>this</code> en clases.
2.RXJS-Javascript/	<code>01_intro.js</code> <code>02_observable.js</code> <code>03_pipes.js</code> <code>04_merge_concat.js</code>	Fundamentos de RxJS: Observable (\$), Observer, suscripción, operadores de creación, pipes, y <code>merge</code> vs <code>concat</code> .

Directorio	Archivos Clave	Temas que Cubre
3.Angular-RXJS/	<code>src/app/models/*</code> <code>app.ts</code> <code>app.html</code> <code>app.css</code>	Aplicación Angular completa consumiendo JSONPlaceholder (users y posts) con <code>mergeMap</code> .

3. Desglose Técnico por Módulos

Parte 1: Funciones en JavaScript (1.Funciones/)

- **Evitar sobreescritura accidental:** Se demuestra cómo declarar funciones con `function sumar()` permite que cualquier parte del programa redefina la función accidentalmente. La buena práctica enseñada es declarar siempre con `const sumar = () => ...`.
- **Sintaxis Arrow Function:**
 - Una línea con retorno implícito (sin llaves ni `return`): `const dividir = (a, b) => a / b;`
 - Un solo parámetro no requiere paréntesis: `const duplicar = a => a * 2;`
 - Cero parámetros exigen paréntesis: `const saludar = () => console.log('Hola');`
- **Alcance léxico de `this` (`arrow_vs_normal.js`):** Se implementó la clase `Animal`. Cuando una función interna dentro de un método se declara con `function()` tradicional, `this` pierde el contexto del objeto y produce error. Cuando se declara con `() => {}`, hereda el `this` del objeto circundante.

Parte 2: Programación Reactiva con RxJS en Node (2.RXJS-Javascript/)

Configurado como paquete ES Module (`"type": "module"`) con dependencia de `rxjs`:

Archivo	Conceptos Implementados	Demostración Clave
<code>01_intro.js</code>	<code>Observable</code> , <code>Observer</code> , convención <code>\$</code> .	Observer como objeto con <code>next</code> , <code>error</code> y <code>complete</code> . Suscripción de múltiples observadores al mismo flujo.
<code>02_observable.js</code>	Operadores de creación: <code>of</code> , <code>from</code> , <code>range</code> , <code>interval</code> .	Creación artesanal <code>handmadeFrom</code> con <code>new Observable</code> demostrando que los operadores son abreviaciones sintácticas.
<code>03_pipes.js</code>	Tuberías: <code>take</code> , <code>map</code> , <code>filter</code> , <code>reduce</code> .	Experimento del video: • <code>take(5) → filter(pares) → map(*2)</code> = <code>[0, 4, 8]</code> (3 datos). • <code>filter(pares) → take(5) → map(*2)</code> = <code>[0, 4, 8, 12, 16]</code> (5 datos). <i>¡El orden de los operadores altera totalmente el resultado!</i>
<code>04_merge_concat.js</code>	Combinación de observables: <code>merge</code> vs <code>concat</code> .	• <code>merge</code> emite de forma intercalada según el tiempo de respuesta. • <code>concat</code> espera a que el primer flujo complete antes de iniciar el segundo.

Caso de Uso Web (Merge vs Concat): Si lanzamos la Petición 1 (Verificar sesión) y la Petición 2 (Cargar perfil con `sessionId`), usar `merge` puede hacer que la Petición 2 responda primero por latencia cuando `sessionId` aún es nulo. `concat` garantiza el orden estricto.

Parte 3: Angular con RxJS y JSONPlaceholder (3.Angular-RXJS/)

Aplicación Angular 19/22 configurada con componentes standalone e inyección moderna de dependencias:

- **Inyección de HTTP:** Se configuró `provideHttpClient()` en `app.config.ts` e inyección con `inject(HttpClient)` en el componente.
- **Modelos de datos:** Interfaces `User` (id, name, username, email...) y `Post` (userId, id, title, body).
- **El problema real resuelto con `mergeMap`:**

El usuario busca por `username`. El servidor requiere primero encontrar el usuario para conocer su `usuario.id`, y únicamente luego consultar `posts?userId=...`.

```
this.http.get<User[]>(`${this.rootURL}/users?username=${usernameBuscado}`)
  .pipe(
    mergeMap(users => {
      if (users && users.length > 0) {
        this.usuario = users[0];
        // Segunda petición dependiente del ID obtenido
        return this.http.get<Post[]>(`${this.rootURL}/posts?userId=${this.usuario.id}`);
      } else {
        this.usuario = null;
        return of([]);
      }
    })
  )
  .subscribe(posts => {
    this.publicacion = posts[0];
  });
```

- **Interfaz de Usuario:** Formulario interactivo, botones de acceso rápido para probar con usuarios existentes (`Bret`, `Antonette`, `Samantha`) y visualización condicional de tarjetas con `@if`.

4. Instrucciones de Ejecución para el Equipo

1. Ejecutar scripts de funciones:

```
cd "TallerRXJS/1.Funciones"
node funciones.js
node arrow_vs_normal.js
```

2. Ejecutar scripts de RxJS:

```
cd "TallerRXJS/2.RXJS-Javascript"
node 01_intro.js
node 02_observable.js
node 03_pipes.js
node 04_merge_concat.js
```

3. Levantar la aplicación de Angular:

```
cd "TallerRXJS/3.Angular-RXJS"
npm start
# Abrir navegador en http://localhost:4200
```

5. Sugerencias y Mejoras para la Próxima Semana (Semana 10)

- **Conexión con Spring Boot:** Reemplazar la URL base de JSONPlaceholder por `http://localhost:8080/api/...` una vez creados los controladores en el backend.
- **Manejo de Errores con `catchError`:** En el taller se usó `of([])` para flujos vacíos; para producción se recomienda agregar `catchError` dentro del `pipe` para capturar respuestas HTTP 404 o 500 sin romper el flujo.